Git Repository Health Dashboard

Team name: Stack Attack

1. What We Have Done:

During this hackathon, we built a Cloud-Based Engineering Telemetry Dashboard. We successfully engineered a full-stack data pipeline that extracts live repository data, mathematically analyzes it for engineering risks, securely stores the historical snapshots in a cloud database, and visualizes the long-term trends. We transformed raw GitHub metadata into actionable enterprise intelligence.

2. The Tech Stack

We carefully selected a modern, highly scalable tech stack to handle the entire data lifecycle:

- ➢ **Frontend & Compute Layer:** Next.js (React) & TypeScript. We used Next.js not just for the beautiful UI, but because its API routes provide a stateless, horizontally scalable compute layer.
- ➢ **Styling:** Tailwind CSS & Lucide Icons. Used to create a premium, dark-mode, enterprise SaaS aesthetic.
- ➢ **Data Source:** GitHub REST API. Used to fetch live telemetry (commits, contributors, pull requests).
- ➢ **Cloud Persistence:** Amazon Web Services (AWS RDS - PostgreSQL). Used as our secure, decoupled data warehouse to store historical health snapshots.
- ➢ **Observability:** Grafana Cloud. Used to query our AWS database and render dynamic, embeddable time-series graphs.

3. Detailed Feature Breakdown

We built a massive suite of features to provide complete visibility into software projects. Here is exactly what we engineered:

Multi-Tenant Repository Selector

- ➢ **What it is:** A dynamic dropdown menu in the top navigation.
- ➢ **What it does:** Allows the user to instantly switch between 9 different repositories across the team (e.g., Discourse, Music-Player, SaaS-UI).
- ➢ **The Tech:** It updates the React state, instantly re-routes the data pipeline to fetch new GitHub data, and dynamically updates the embedded Grafana charts without requiring a page reload.

The Deterministic Rules Engine

- ➢ **What it is:** Our custom-built, algorithmic analysis engine.
- ➢ **What it does:** Instead of using Generative AI (which can hallucinate or suffer from API latency), we hardcoded strict engineering heuristics. It analyzes the raw data and calculates a highly accurate 0-100 Health Score based on penalty formulas for high code churn or project stagnation.
- ➢ **The Output:** It generates a written, paragraph-length analysis explaining the exact state of the repository.

Visual Rules Engine Alerts

- ➢ **What it is:** A stack of dynamic notification boxes below the Rules Engine analysis.
- ➢ **What it does:** It breaks the algorithmic analysis down into actionable UI alerts. It evaluates specific thresholds and renders green Success boxes (e.g., healthy bus factor), yellow Warning boxes, or red Error boxes (e.g., critical stagnation).

Key Engineer Profiler

- ➢ **What it is:** A dedicated UI card that identifies the project's single point of failure.
- ➢ **What it does:** By mapping over the GitHub contributor array, the backend identifies the developer with the most commits. It then dynamically renders their GitHub avatar and calculates their total commit share. If their share is dangerously high, it flags them as a "Bottleneck," warning management of a high "Bus Factor" risk.

Animated SVG Health Gauge

- ➢ **What it is:** A custom-built SVG radial progress chart.
- ➢ **What it does:** Takes the 0-100 score from the Rules Engine and animates a circular stroke. It dynamically changes colors based on the score (Green for >80, Yellow for >60, Red for failing), giving executives a 1-second visual summary of the project's status.

Live Metric Cards Grid

- ➢ **What it is:** Four real-time telemetry readouts.
- ➢ **What it does:** Displays the raw numbers powering the engine:

- ➔ **Code Churn:** Total lines changed (identifies messy or overly complex commits).
- ➔ **Stagnation Risk:** Days since the last commit (identifies dead projects).
- ➔ **Bus Factor:** The percentage of code owned by the top contributor.
- ➔ **PR Bottlenecks:** The number of currently open pull requests.

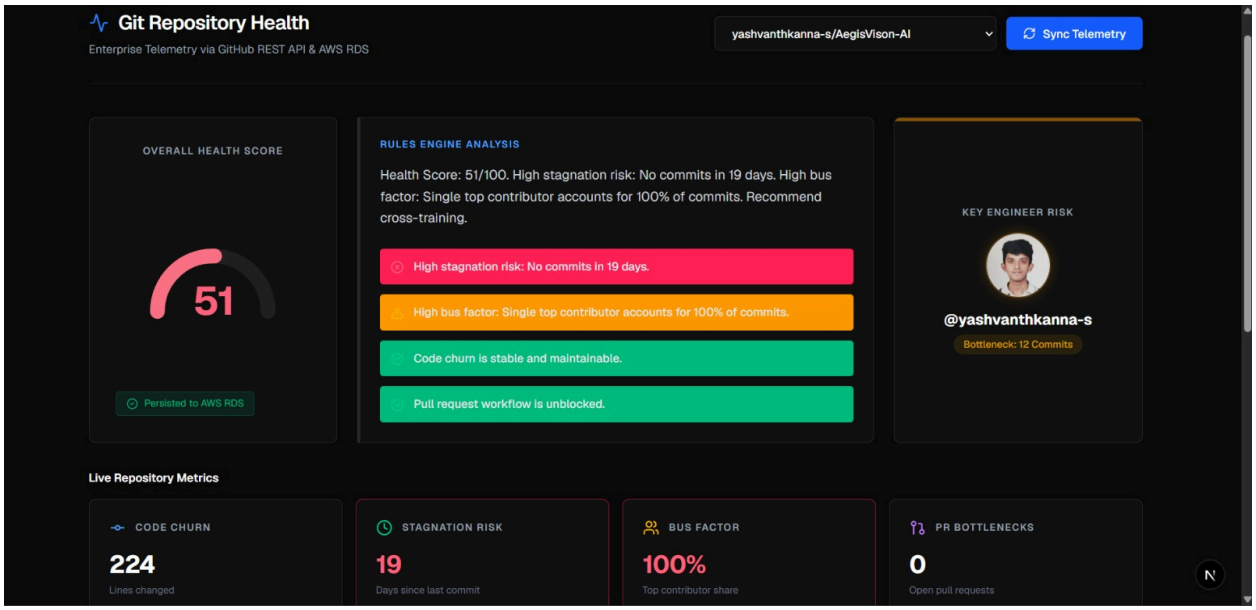
AWS RDS Persistence Pipeline

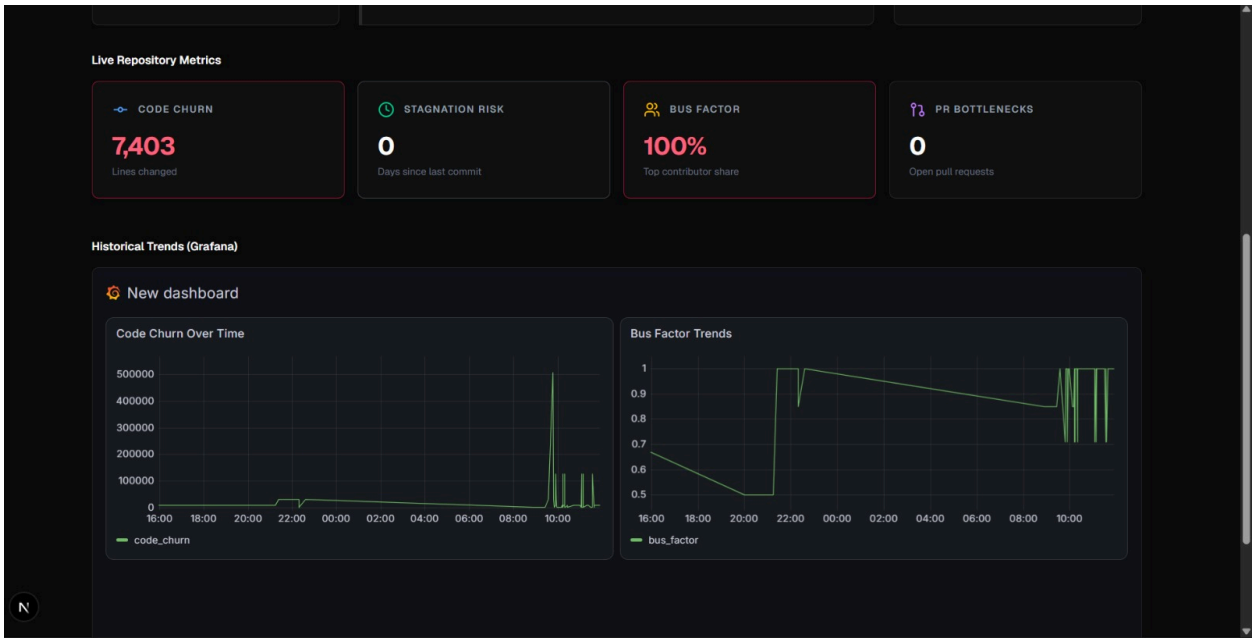
- **What it is:** Our cloud-based data storage mechanism.
- **What it does:** Every single time a user clicks "Sync Telemetry," the Next.js backend opens a secure PostgreSQL connection to Amazon AWS. It inserts a new row containing the repo_name, the exact timestamp, the calculated score, and the raw metrics. A small green checkmark appears on the dashboard to confirm the data was successfully vaulted in the cloud.

Embedded Grafana Observability

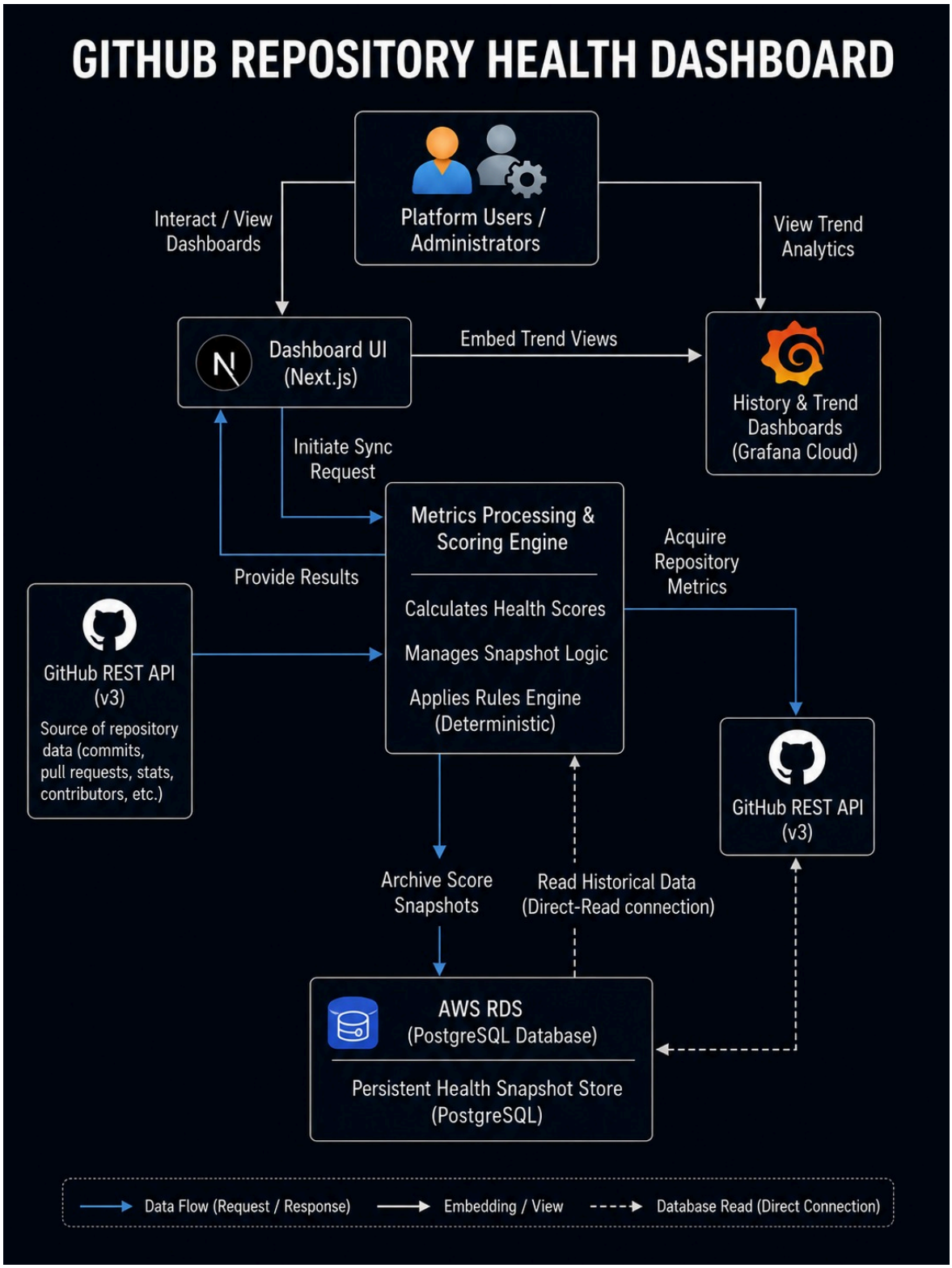
- **What it is:** Side-by-side time-series graphs embedded directly into the Next.js UI.
- **What it does:** We securely embedded Grafana d-solo panels into the dashboard. When you select a repository in Next.js, it passes a dynamic variable to Grafana. Grafana then queries the AWS database and instantly renders historical trend lines for both Code Churn and Bus Factor, allowing management to see if a project's health is improving or degrading over time.

Git Health Dashboard:





Solution Architecture:



Architecture Flow:

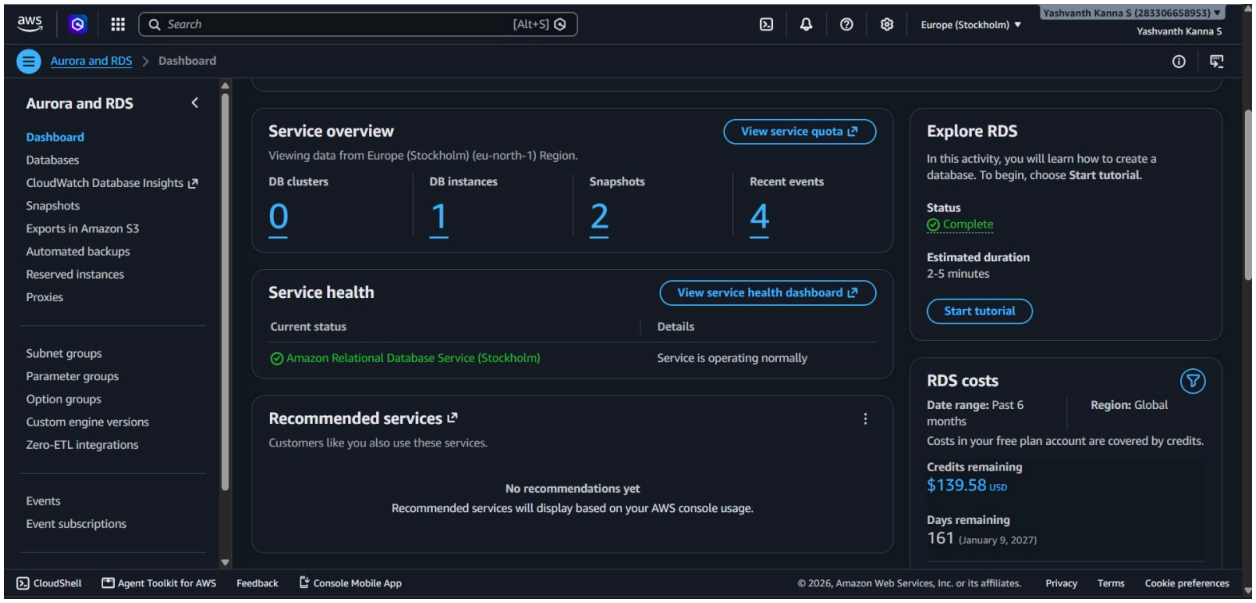
1. Historical Trend Analysis (The Time Machine Effect) GitHub’s API is great at telling you what a repository looks like right now, but it cannot easily tell you what the health score was 3 months ago. By vaulting our calculated health scores, bus factors, and code churn metrics into AWS RDS every time we sync, we are building a permanent, queryable history. AWS RDS is what allows Grafana to draw trendlines over time, allowing managers to see if a team is improving or heading towards burnout.

2. Bypassing API Rate Limits (High Availability) Enterprise engineering teams pull massive amounts of data. If our dashboard queried GitHub directly every time a user loaded a graph, we would instantly hit GitHub's API rate limits and the dashboard would crash. By using AWS RDS as our central source of truth, we only query GitHub once during a sync. From that point on, our dashboard and Grafana read directly from our highly available AWS database, guaranteeing 100% uptime and blazing-fast load speeds for the user.

3. Enterprise-Grade Security and Scalability We chose AWS RDS because it is the industry standard for enterprise data. The connection between our Next.js backend and the AWS database is secured via SSL encryption. Furthermore, as our tool scales to track thousands of repositories across a massive company, AWS RDS allows us to scale our compute and storage infinitely with zero downtime.

"GitHub gives us the raw data, but AWS RDS gives us the memory. Without AWS RDS, our graphs wouldn't exist, our historical trends would be lost, and our dashboard would constantly crash from API limits. RDS makes this a true enterprise product."

AWS RDS :


aws
Search
[Alt+S]
Europe (Stockholm)
Yashvanth Kanna S (283306658953)
Yashvanth Kanna S
Aurora and RDS
Dashboard
Aurora and RDS
Dashboard
Databases
CloudWatch Database Insights
Snapshots
Exports in Amazon S3
Automated backups
Reserved instances
Proxies
Subnet groups
Parameter groups
Option groups
Custom engine versions
Zero-ETL integrations
Events
Event subscriptions
Service overview
View service quota
Viewing data from Europe (Stockholm) (eu-north-1) Region.
DB clusters
0
DB instances
1
Snapshots
2
Recent events
4
Service health
View service health dashboard
Current status
Details
Amazon Relational Database Service (Stockholm)
Service is operating normally
Recommended services
Customers like you also use these services.
No recommendations yet
Recommended services will display based on your AWS console usage.
Explore RDS
In this activity, you will learn how to create a database. To begin, choose Start tutorial.
Status
Complete
Estimated duration
2-5 minutes
Start tutorial
RDS costs
Date range: Past 6 months
Region: Global
Costs in your free plan account are covered by credits.
Credits remaining
$139.58 USD
Days remaining
161 (January 9, 2027)
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences